Щучкин Максим, Б07-202

Отчет по лабораторной работе #4

«Решение задачи численного интегрирования в сетевой модели»

Цель: освоить гибридную модель параллелизма MPI+OpenMP для распределённых вычислений. Реализовать алгоритм суммирования результатов по методу сдваивания.

Текст программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <mpi.h>
#include <omp.h>

#define TAG 777

/* Интегрируемая функция */
double f1(double x) { return 4.0 / (1.0 + x * x); }

/* Метод средних прямоугольников с OpenMP-редукцией */
double integrate(double a, double b, int n) {
    double h = (b - a) / n;
    double s = 0.0;
    #pragma omp parallel for reduction(+:s)
    for (int i = 0; i < n; i++) {
        s += f1(a + h * (i + 0.5));
    }
    return h * s;
}

int main(int argc, char *argv[]) {
    int np, mp, nt = 1;
    if (argc > 1) nt = atoi(argv[1]);

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &np);
    MPI_Comm_rank(MPI_COMM_WORLD, &mp);
    omp_set_num_threads(nt);

    const int TOTAL_N = 1000000000;
    double a = 0.0, b = 1.0;
    double h_proc = (b - a) / np;
    double a1 = h_proc * mp;
    double b1 = (mp == np - 1) ? b : a1 + h_proc;
    int n1 = TOTAL_N / np;
```

```
    /* Замер времени вычислений */
    double t_start = MPI_Wtime();
    double my_sum = integrate(a1, b1, n1);
    double t_comp = MPI_Wtime() - t_start;

    /* Замер времени редукции методом сдваивания */
    t_start = MPI_Wtime();
    int step = 1;
    double recv_val;
    MPI_Status status;

    while (step < np) {
        if (mp % (2 * step) == 0) {
            if (mp + step < np) {
                MPI_Recv(&recv_val, 1, MPI_DOUBLE, mp + step, TAG,
MPI_COMM_WORLD, &status);
                my_sum += recv_val;
            }
        } else {
            if (mp - step >= 0) {
                MPI_Send(&my_sum, 1, MPI_DOUBLE, mp - step, TAG,
MPI_COMM_WORLD);
                break;
            }
        }
        step *= 2;
    }
    double t_comm = MPI_Wtime() - t_start;
    double t_total = t_comp + t_comm;

    fprintf(stderr, "mp=%d t1=%10lf t2=%10lf t3=%10lf int=%22.15le\n",
            mp, t_comp, t_comm, t_total, my_sum);

    MPI_Finalize();
    return 0;
}
```

Output:

```
fnbik09@sr210:~/mpc/task4$ mpicc -fopenmp -O2 solution_04.c -o solution_04.px -lm
fnbik09@sr210:~/mpc/task4$ mpirun -np 1 solution_04.px 1 2>&1 | grep -v Authorization
mp=0 t1=  1.670198 t2=  0.000000 t3=  1.670198 int= 3.141592653589971e+00
fnbik09@sr210:~/mpc/task4$ mpirun -np 8 solution_04.px 2 2>&1 | grep -v Authorization
mp=3 t1=  0.140024 t2=  0.000028 t3=  0.140052 int= 4.195077549209431e-01
mp=7 t1=  0.153420 t2=  0.000016 t3=  0.153436 int= 2.662726551032794e-01
mp=1 t1=  0.163719 t2=  0.000021 t3=  0.163740 int= 4.824946743203739e-01
mp=5 t1=  0.164992 t2=  0.000021 t3=  0.165014 int= 3.396071737989367e-01
mp=2 t1=  0.175744 t2=  0.000030 t3=  0.175773 int= 8.746757834957922e-01
mp=0 t1=  0.165808 t2=  0.037628 t3=  0.203436 int= 3.141592653589833e+00
mp=4 t1=  0.141729 t2=  0.061753 t3=  0.203481 int= 1.287002217586612e+00
mp=6 t1=  0.203152 t2=  0.000032 t3=  0.203184 int= 5.675882184166453e-01
fnbik09@sr210:~/mpc/task4$
```

Выводы:

1) Точность вычислений: во всех запусках получено int ≈ 3.141592..., что совпадает с аналитическим значением π с малой погрешностью - алгоритм интегрирования и редукции работают без накопления ошибок округления.
2) При переходе с 1 процесс × 1 поток на 8 процессов × 2 потока время вычислений t1 сократилось с ~1.67с до ~0.16с (ускорение ~10x), что подтверждает корректное распределение нагрузки между MPI-доменами и OpenMP-нитями.
3) Время обменов t2 на корневом процессе составило ~0.037с, что существенно меньше, чем при линейном сборе (где t2 росло бы пропорционально np). Логарифмическая сложность редукции успешно реализована.